CS-4112: Introduction to Machine Learning

Convolutional Neural Networks

Arbish Akram

Department of Computer Science
Government College University

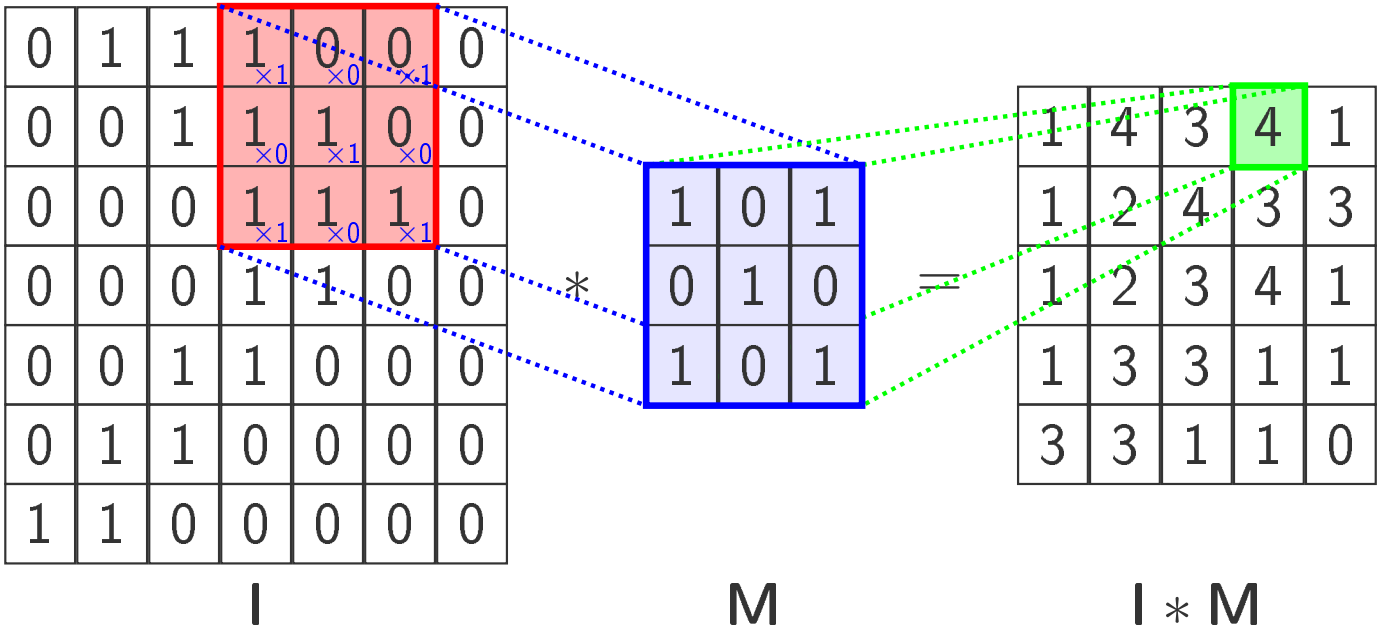
Slides by Dr. Nazar Khan

Convolution

Source: http://www.texample.net/tikz/examples/convolution-of-two-functions/

2D Convolution

Example



Modified from `https://github.com/PetarV-/TikZ/tree/master/2D%20Convolution`

M is usually called a *mask* or *kernel* or *filter*.

Dealing with boundaries

- What about edge and corner pixels where the mask goes outside the image boundaries?
  - Expand image I with virtual pixels. Options are:
    1. Fill with a particular value, *e.g.* zeros.
    2. Replicating boundaries: fill with nearest pixel value.
    3. Reflecting boundaries: mirror the boundary
  - Fatalism: just ignore them. Not recommended since size of $I * M$ will shrink.

Dealing with boundaries

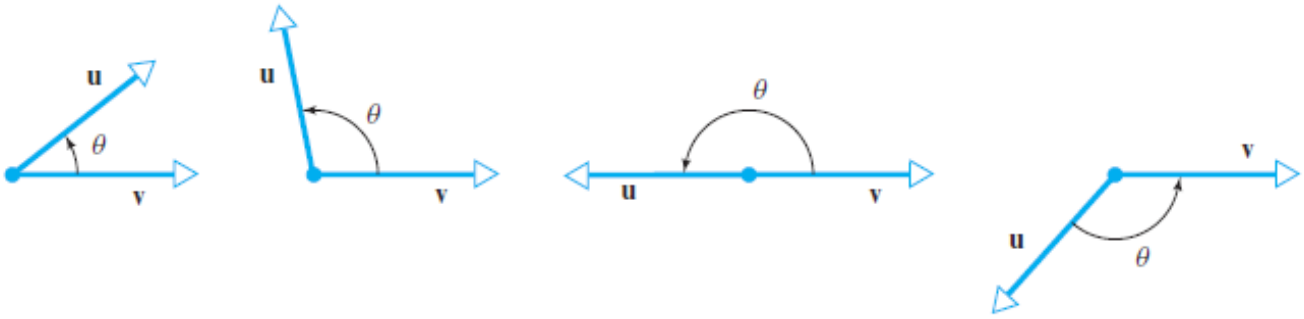
Expand by zeros

For a 5×5 image and 5×5 mask

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & a & b & c & d & e & 0 & 0 \\ 0 & 0 & f & g & h & i & j & 0 & 0 \\ 0 & 0 & k & l & m & n & o & 0 & 0 \\ 0 & 0 & p & q & r & s & t & 0 & 0 \\ 0 & 0 & u & v & w & x & y & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

A Neuron as a Detector

- A neuron can be viewed as a detector.
- When it fires, the input must have been similar to its weights.
  - Firing $\implies$ $\mathbf{w}^T\mathbf{x}$ was high $\implies$ $\mathbf{w}$ was similar to $\mathbf{x}$
- So neuron firing indicates detection of something similar to its weights.



$$\mathbf{u}^T\mathbf{v} = \|\mathbf{u}\|\|\mathbf{v}\| \cos\theta$$

- Since $-1 \leq \cos\theta \leq 1$, $\mathbf{u}^T\mathbf{v}$ is highest when $\cos\theta = 1$
- That happens when $\theta = 0$
- That happens when vectors $\mathbf{u}$ and $\mathbf{v}$ point in the same direction.

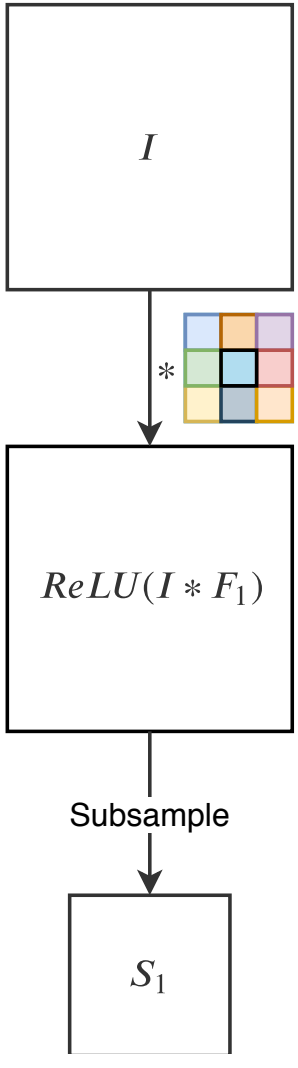
Convolutional Neural Networks

- Now we will look at networks that produce neuronal output via convolution.
- Known as Convolutional Neural Networks (CNNs).
- Most frequently used network architecture.
- Exploits local correlation of inputs.

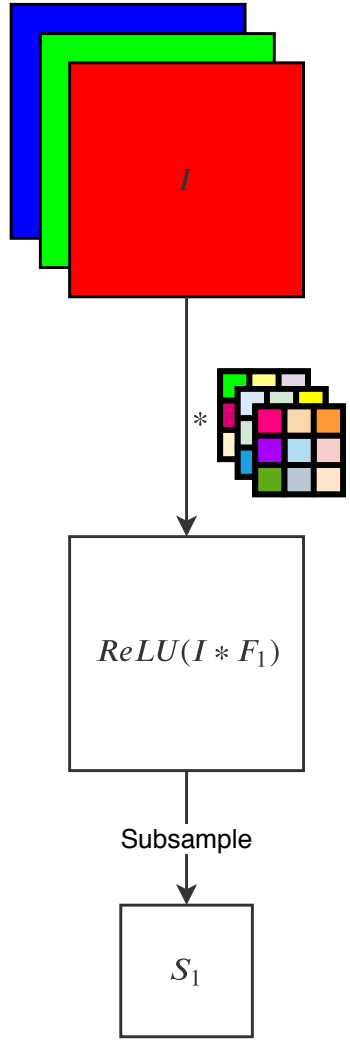
Building Blocks of CNNs

Viewing convolution as neurons

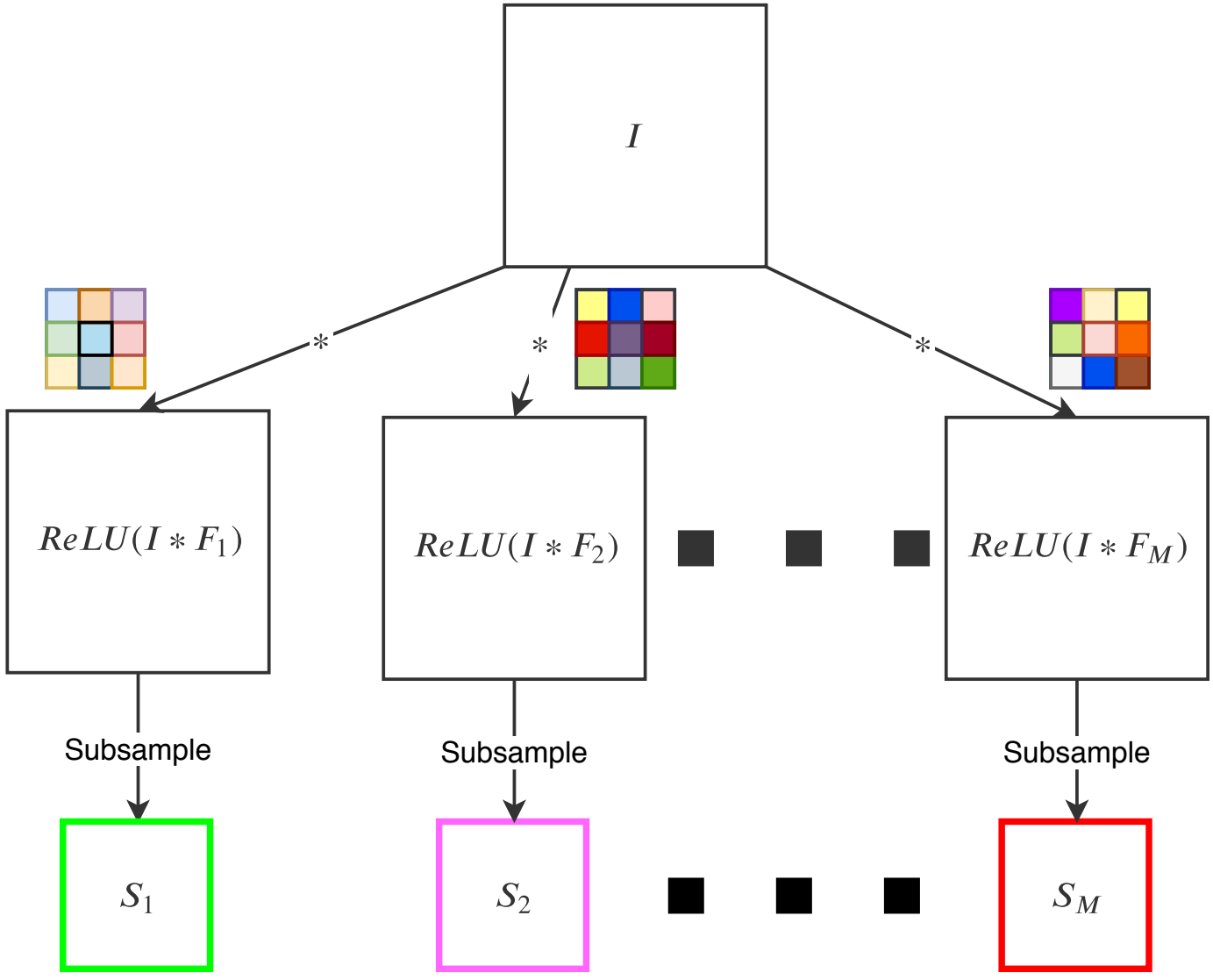
Single channel input



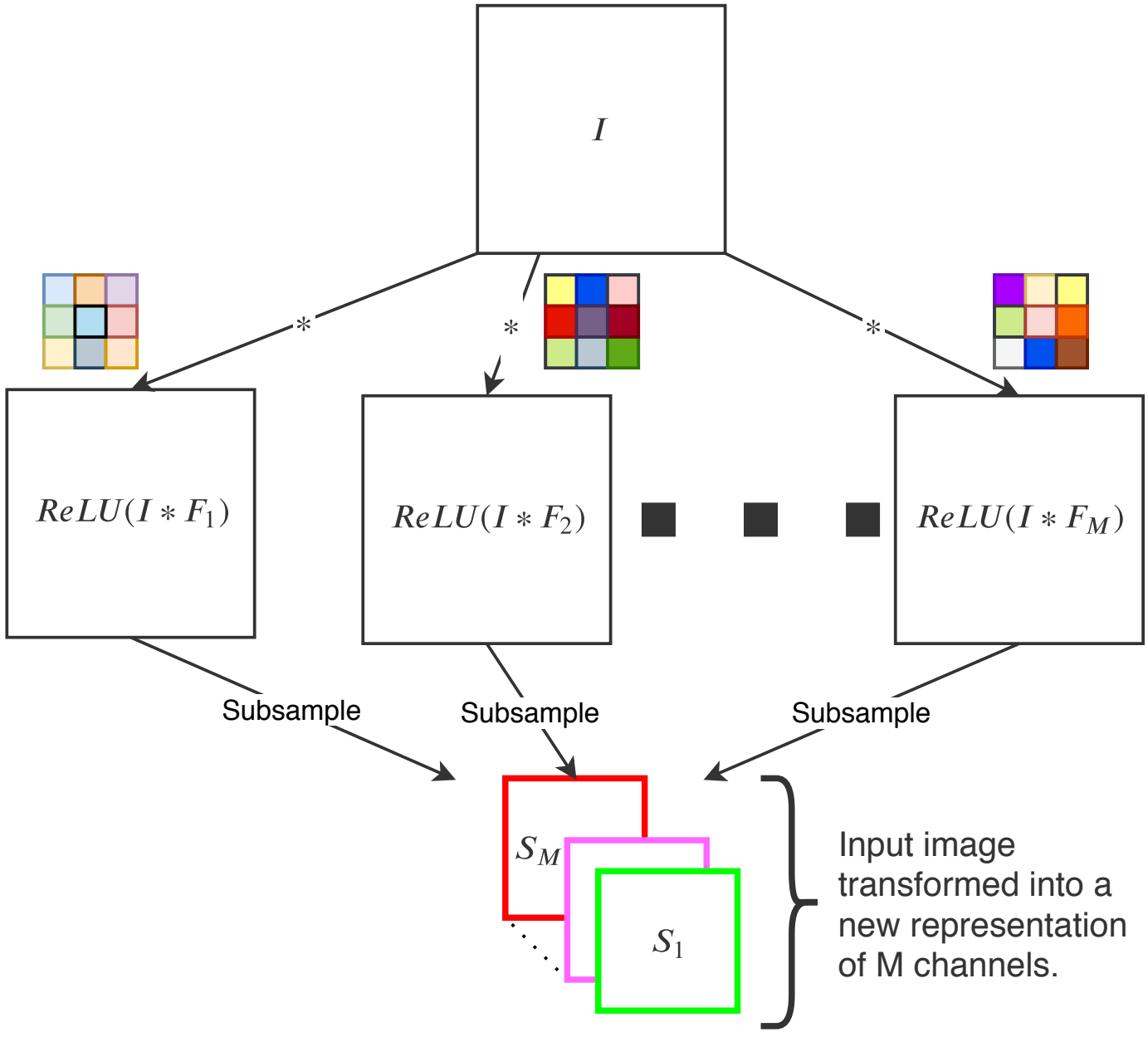
Multichannel input



Building blocks of CNNs

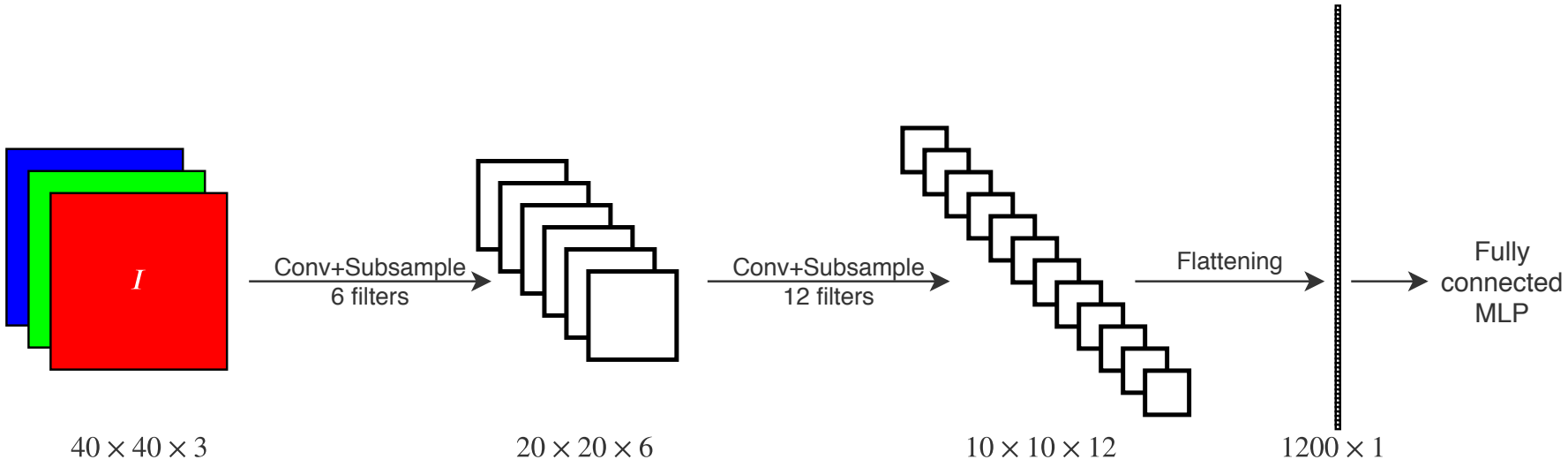


Building blocks of CNNs



CNN

- Convolution by M filters produces M channels.
- They represent an M-channel transformation of the input image I.
- This M-channel image can now be transformed further via additional convolution filters.
- Convolution-subsampling block can be repeated multiple times.
- $I \to M_1$ channels $\to M_2$ channels $\to \cdots \to M_b$ channels $\to$ flattening $\to$ MLP.



Convolutional Neural Networks

- For recognition of hand-written digits, we have seen that inputs are images and outputs are posterior probabilities $p(C_k|\mathbf{x})$ for $k = 1, \ldots, 10$.
- The digits true identity is invariant under
  - translation, scaling, (small) rotation, and
  - small elastic deformations (multiple writings of the same digit by the same person will have subtle differences).
- The output of the neural network should also be invariant to such changes.
- A traditional fully connected neural network can, in principle, learn these invariances using lots of examples.
- However, it totally ignores the *local correlation* property of images.
  - Nearby pixels are more strongly correlated than pixels that are far apart.

Convolutional Neural Networks

- Modern computer vision exploits local correlation by extracting features from local patches and combining this information to detect higher-order features.
  - Example: Gradients $\longrightarrow$ Edges $\longrightarrow$ Lines $\longrightarrow \ldots$.
- Local features useful in one sub-region can be useful in other sub-regions.
  - Example: same object appearing at different locations.
- This weakness of standard neural nets is overcome by CNNs.

NN vs. CNN



- Global receptive fields due to being fully connected.
- Separate weights for each neuron.

- *Local receptive fields* due to being sparsely connected.
- *Shared weights* among different neurons.
- *Subsampling* of each layer's outputs.

- Receptive field of a neuron consists of previous layer neurons that it is connected to (or looking at).

Convolutional layer

- Consists of multiple arrays of neurons. Each such array is called a *slice* or more accurately *feature map*.
- Each neuron in a feature map
  - is connected to only few neurons in the previous layer, but
  - uses the same weight values as all other neurons in that feature map.
- So within a feature map, we have both
  - local receptive fields, and
  - shared weights.

Convolutional layer

- Example: A feature map may have
  - 100 neurons placed in a 10×10 array, with
  - each neuron getting input from a 5×5 patch of neurons in the previous layer (receptive field), and
  - the same $26 (= 5 \times 5 + 1)$ weights shared between these 100 neurons.
- **Viewed as detectors, all** 100 **neurons detect the same** 5×5 **pattern but at different locations of the previous layer**.
- Different feature maps will learn[1] to detect different kinds of patterns.
  - For example, one feature map might learn to detect horizontal edges while others might learn to detect vertical or diagonal edges and so on.

[1] based on their learned weights

Convolutional layer

- To compute activations of the 100 neurons, a dot-product is computed between the same shared weights and different 5×5 patches of previous layer neurons.
- This is equivalent to **sliding a window of weights over the previous layer and computing the dot-product at each location of the window**.
- Therefore, activations of the feature map neurons are computed via *convolution* of the previous layer with a *kernel* comprising the shared weights. Hence the name of this layer.

Subsampling layer

- Reduces the spatial dimensions of the previous layer by downsampling. Also called *pooling* layer.
- Example: downsampling previous layer of $n \times n$ neurons by factor 2 yields a pooled layer of $\frac{n}{2} \times \frac{n}{2}$ neurons.
- No adjustable weights. Just a fixed downsampling procedure.
- Reduces computations in subsequent layers.
- Reduces number of weights in subsequent layers. This reduces overfitting.

Subsampling

- Options: From non-overlapping 2×2 patches
  - pick top-left (standard downsampling by factor 2)
  - pick average (*mean-pooling*)
  - pick maximum (*max-pooling*)
  - pick randomly (*stochastic-pooling*)
- Fractional max-pooling: pick pooling region randomly.

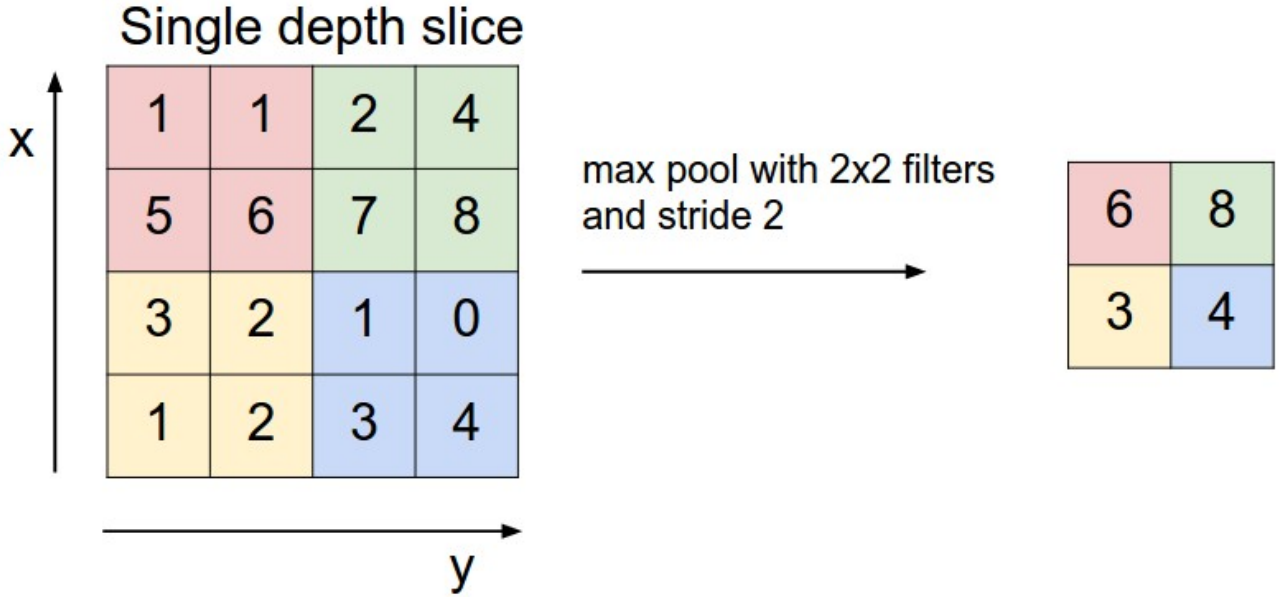


Figure: Max-pooling with 2×2 receptive fields, and stride of 2 neurons. Source: `http://cs231n.github.io/convolutional-networks/`

Subsampling

- The options in the last slide discard 75% of the data.
- They correspond to
  - neurons with 2×2 receptive fields, and
  - *stride* of 2 neurons.
- This is the most commonly used configuration. Other options exist but note that pooling with larger receptive fields discards too much data.
- Subsampling layer can be skipped if convolution layers uses stride>1 since that also produces a subsampled output.

Subsampling

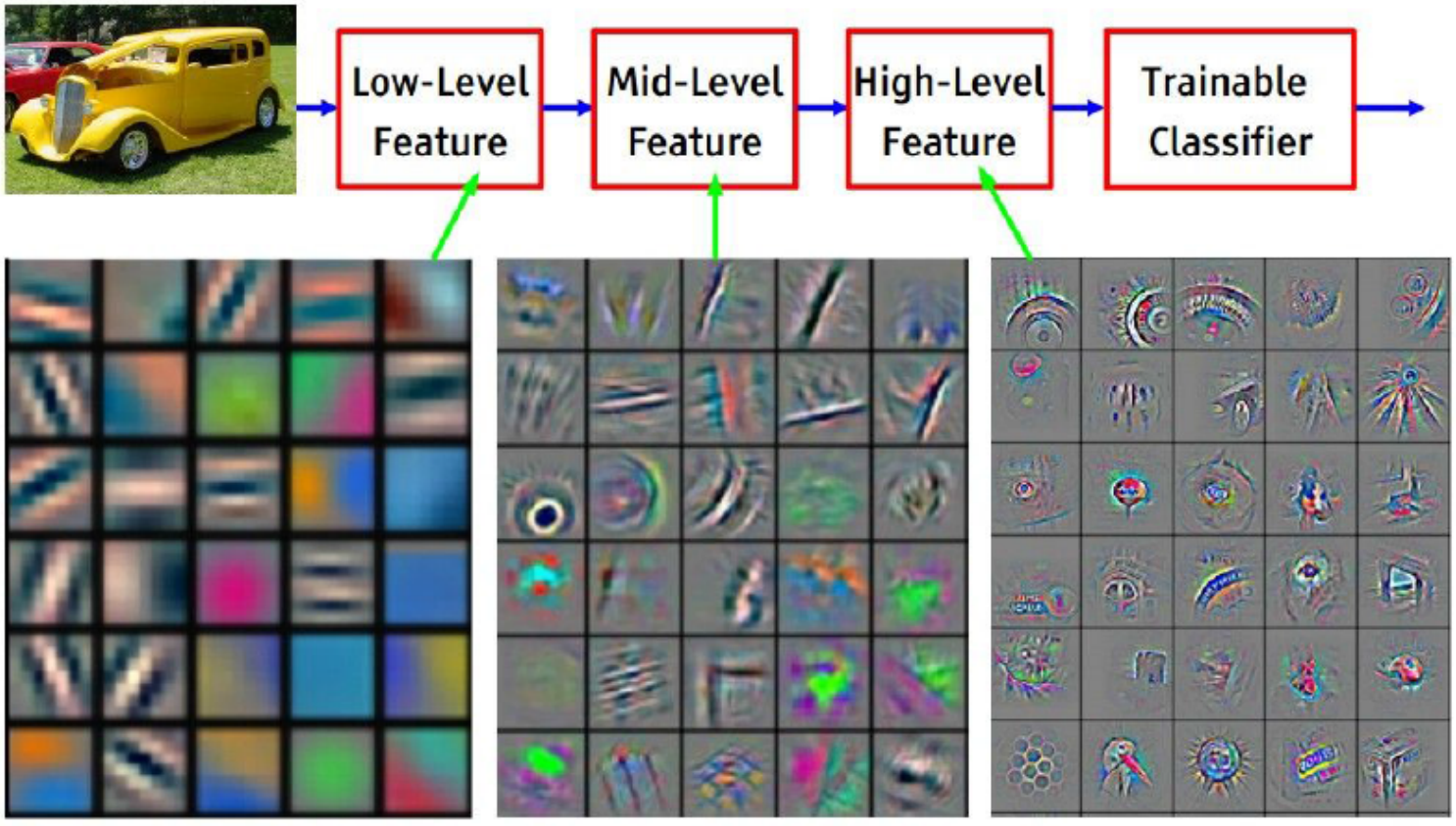
A pooling layer

- with $F \times F$ receptive field and stride S,
- "looking at" a $W_1 \times H_1 \times D_1$ input volume,
- produces a $W_2 \times H_2 \times D_2$ output volume, where
  - $W_2 = \frac{W_1 - F}{S} + 1$
  - $H_2 = \frac{H_1 - F}{S} + 1$
  - $D_2 = D_1$.

Fully Connected Layers

- After flattening, a fully connected MLP can be used.
- The last layer has
  - neurons equal to the desired output size, and
  - activation functions based on the problem to be solved.
- The flattened layer can therefore be viewed as a transformation $\phi(\mathbf{x})$ that is fed into an MLP.
- Similarly, outputs of earlier layers are *intermediate representations* of the input.

Intermediate Representations



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Intermediate feature representations. Early layers form simple, low-level representations of the input. They are used to incrementally form more complex, high-level representations.

Source: `http://cs231n.stanford.edu/slides/winter1516_lecture7.pdf`

CNN Variations

- There are *lots* of variations.
  - Fully convolutional networks. No pooling and no fully connected layer.
  - 1×1 convolutions to reduce computations.
  - Inception modules to combine multiple filter sizes.
  - Residual blocks to avoid vanishing gradients.
  - Depthwise separable convolutions to reduce parameters and computations.
  - Lightweight and fast models (SqueezeNet, MobileNet, . . .) for edge computing.
  - Fast search over hyperparameters (EfficientNet).

Summary

- Many signals such as sounds, images, and videos obey the *local correlation property*.
- MLPs ignore local correlation.
- CNNs are a special kind of neural architecture that
  - exploits local correlation
  - extracts multi-scale features, and
  - extracts *translation covariant* features
- These properties are built into the design of the architecture.